

Chess Coach Agent — How It Works

A Teaching Guide for the Developer

What Is This Project?

Chess Coach Agent is a multi-agent AI system that analyzes your Chess.com games and produces a personalized coaching plan. Three specialized AI agents — each powered by Claude — collaborate in a pipeline. Each agent has one job, and they pass structured information between each other through a shared 'state' object.

What Is LangGraph?

LangGraph is a library built on top of LangChain that lets you define AI workflows as a directed graph. Think of it like a flowchart where each box is a function (called a node) and the arrows between boxes are edges. The special thing about LangGraph is that the graph can have loops — so an agent can run, get feedback, and run again until the output is good enough.

In this project, the Coach node and Critic node form a feedback loop: if the Critic scores the coaching report below 70/100, LangGraph sends control back to the Coach with the critic's feedback included. This continues up to 3 times.

The Three Agents

Agent 1: The Analyst

The Analyst fetches your recent games from Chess.com's public API, parses the PGN move notation using the python-chess library, and sends all the game data to Claude with a specific prompt asking for pattern recognition across games — not a move-by-move analysis of one game, but themes that repeat.

A key feature: the Analyst marks moves made under time pressure (less than 30 seconds on the clock) with a warning. The Coach is then instructed to deprioritize mistakes on time-pressured moves, because rushing errors are different from strategic misunderstandings.

Agent 2: The Coach

The Coach has two steps. First, it interviews you — asking about your goals and which openings you want to play. This matters because a player who wants to learn the Sicilian should get different advice than one who plays 1...e5. Second, it generates a full coaching plan using Claude, incorporating the Analyst's findings AND your stated goals.

If the Critic rejects the report, the Coach node runs again — but this time the state includes the critic's feedback, so Claude knows exactly what to fix.

Agent 3: The Critic

The Critic is Claude acting as a quality-control judge. It reads both the original game analysis AND the coaching report and scores it across four dimensions: specificity, goal alignment, actionability, and time-pressure handling. It outputs structured JSON with a total score. Below 70 = rejection with specific feedback. Above 70 = approved.

The Critic pattern is important in production AI systems. LLMs can produce plausible-sounding but vague or generic output. Having a second LLM judge the first's work with explicit criteria improves quality without human review on every run.

How State Works in LangGraph

All agents share a single Python dictionary called `ChessCoachState` (defined as a `TypedDict`). When a node runs, LangGraph passes the full current state to it. The node returns only the keys it wants to update. LangGraph merges those updates back into the state before passing it to the next node. No node sees stale data — the state is always current.

User Memory

When you link your Chess.com username, it's saved to a local SQLite database. On future runs, the system recognizes you and pre-fills your preferences (time control, openings, goals). This is not cloud storage — it lives in the `data/` folder on your machine. Chess.com's API is fully public, so no OAuth or tokens are needed.

The Output

After the Critic approves the coaching report, the final output is saved two ways: a Markdown file (readable in any text editor or GitHub) and a PDF (this format) in the `outputs/` folder. The filename includes your username and timestamp so sessions don't overwrite each other.

Version History

V1.0 — Initial release. Full CLI pipeline with all three agents.

Future versions will add a web UI and more detailed game analysis.